20 min read

Policy Gradients: REINFORCE and Actor-Critic

RL Part 7: Learning the policy directly, from REINFORCE to actor-critic.

Recap

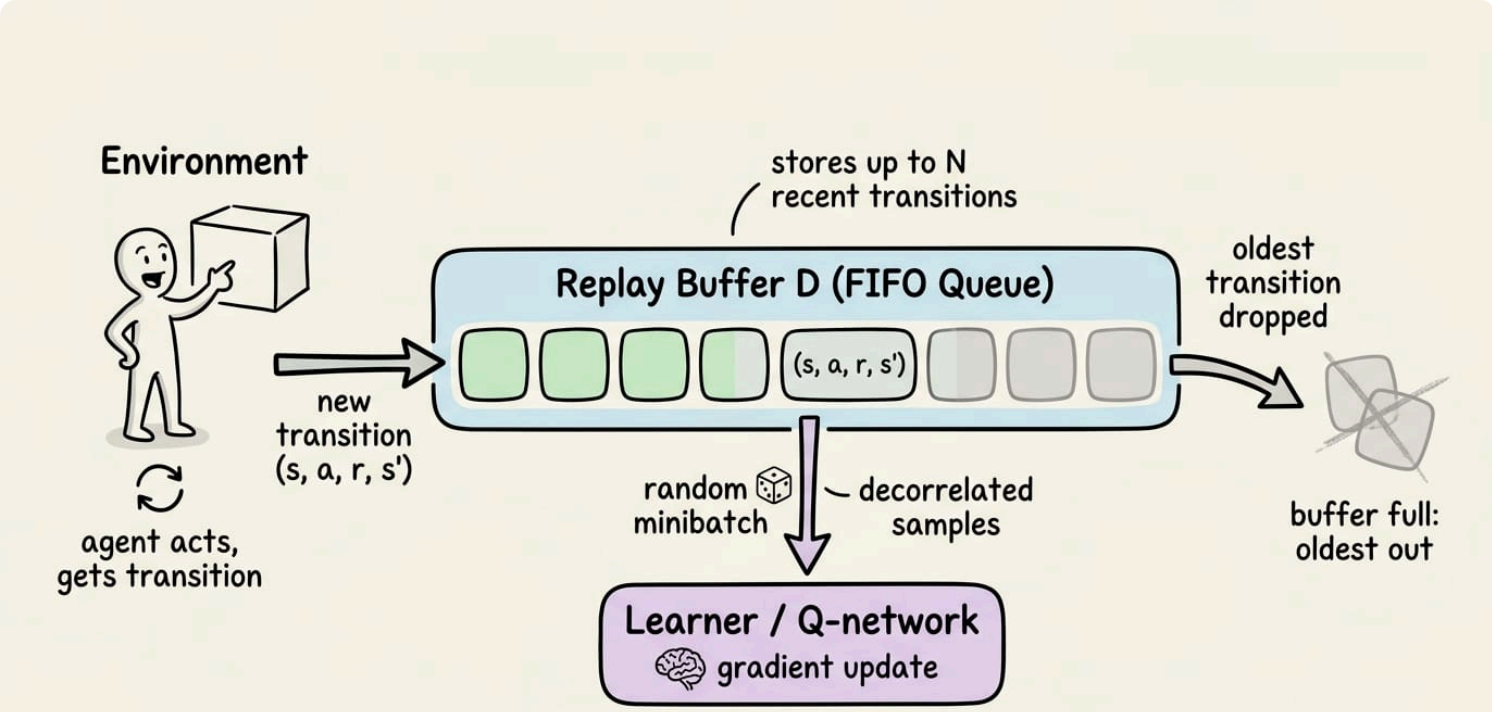
In chapter 6, we moved from linear value function approximation to neural networks.

The value function generalized from a linear form to a neural network. What we gave up in that move was the convergence guarantee that held for linear on-policy methods.

$$\hat{q}(s, a, \theta) = f_\theta(s, a)$$

We then saw what breaks when deep networks meet online Q-learning: correlated samples, moving targets, and the deadly triad scaled up. Two engineering fixes carried the day:

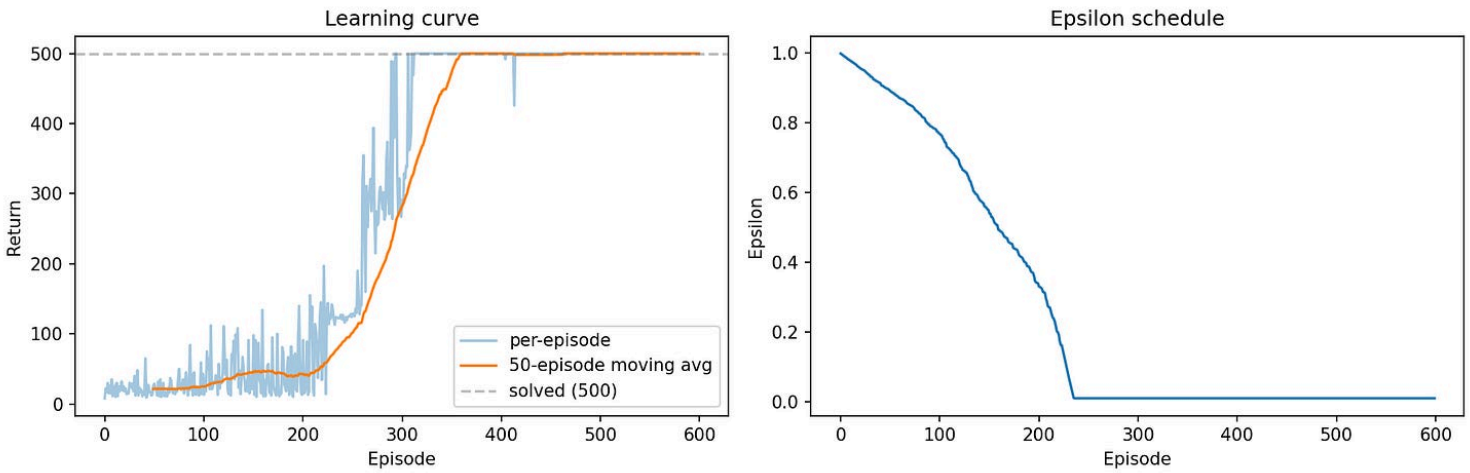
- Experience replay stored past transitions and sampled them in random minibatches, breaking correlation.



- Target networks froze a copy of the Q-network to compute stable bootstrap targets.

$$R_{t+1} + \gamma \max_{a'} Q_{\theta^-}(S_{t+1}, a')$$

Putting these together gave us DQN algorithm, which we trained from scratch on CartPole.

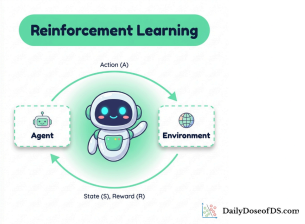


If you have not read Chapter 6, we recommend doing so first:

Introduction to Deep RL and DQN

RL Part 6: From linear features to neural networks, and the engineering choices that makes deep value-based RL possible.

Daily Dose of Data Science • Avi Chawla



Now most of the methods we learned so far share one trait. They learn how good actions are, then derive behavior by picking the best-scoring action. That works beautifully when actions are few and discrete. It strains badly when actions are continuous or numerous.

Hence, there is another way. Instead of scoring actions and choosing among them, we can learn the choosing itself. That is the subject of this chapter.

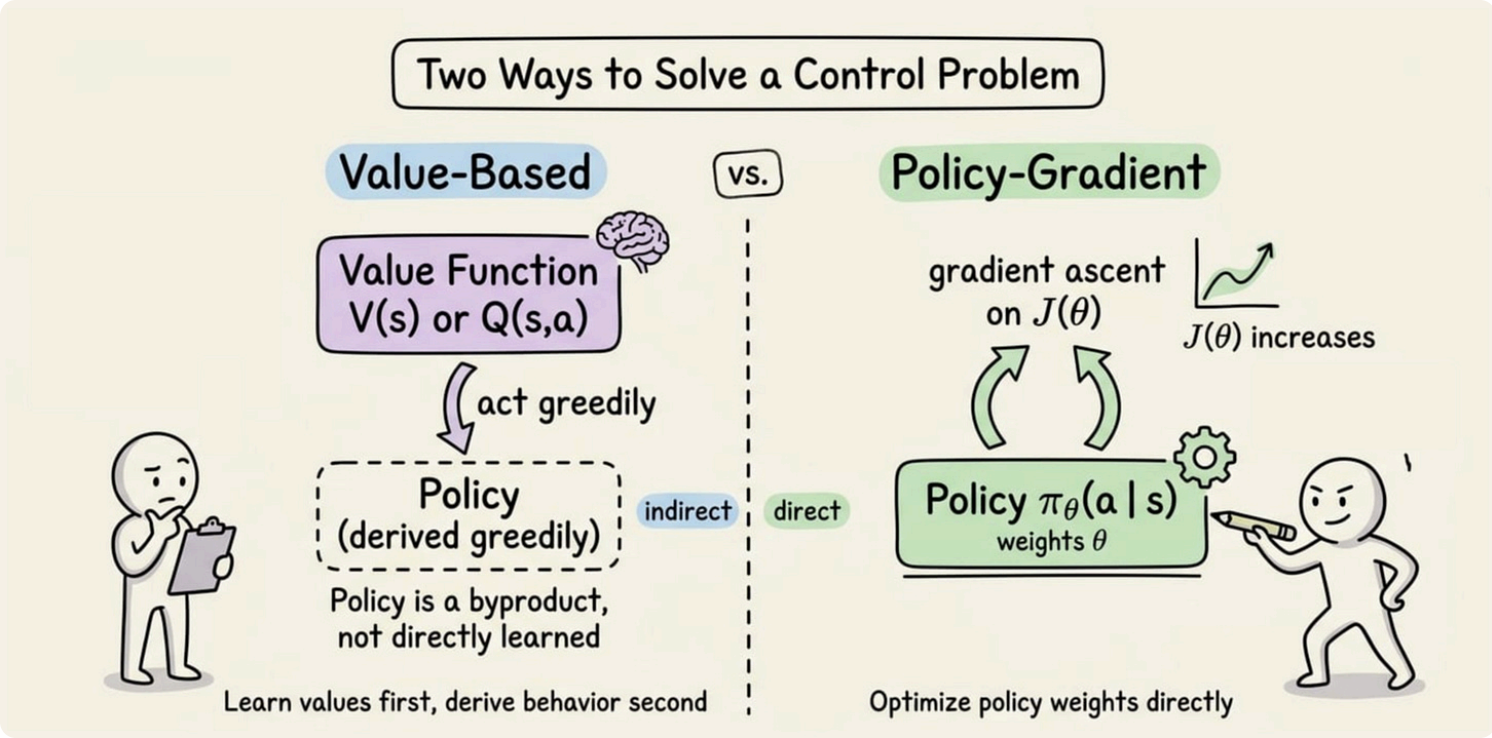
As for this part, we will focus on policy gradient methods. We will build the policy gradient from first principles, meet REINFORCE, confront its variance problem head-on, and learn about the actor-critic architecture.

Let's begin!

Two ways to solve a control problem

A reinforcement learning agent needs a policy: a rule that says what to do in each state. Up to now, we built that rule indirectly. We learned a value function, an estimate of how much reward we can expect, and then acted greedily with respect to it. The policy was a byproduct of the values.

Policy gradient methods flip this around. They parameterize the policy directly and learn its parameters. We write the policy as a function with its own weights, and we adjust those weights to get better behavior.



Let us make this concrete. A policy maps a state to a distribution over actions. We denote it as follows:

$$\pi_\theta(a \mid s)$$

Here π is the policy, θ are its learnable parameters (the weights of a neural network), s is the current state, and a is an action.

👉 The expression $\pi_\theta(a \mid s)$ reads as the probability of taking action a given that we are in state s, under the policy defined by parameters θ.

This is a stochastic policy and for continuous actions, the network typically outputs the mean and spread of a Gaussian distribution, and we sample from it.

👉 An important point or trade-off, however, is that policy gradient methods are on-policy and tend to be sample-hungry. Each update uses data from the current policy, and once we update, that data is stale.

To learn the policy directly, we need an objective: a single number measuring how good the policy is, which we can then push upward. The natural choice is the expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[R(\tau)\right]$$

In this expression:

- $J(\theta)$ is the objective we want to maximize, written as a function of the policy parameters θ.
- The symbol τ (tau) denotes a trajectory, the full sequence of states and actions in an episode.
- The notation $\tau \sim \pi_\theta$ means the trajectory is generated by following the current policy.
- The term $R(\tau)$ is the total return collected along that trajectory.
- $\mathbb{E}[\cdot]$ is the expectation, the average over all the trajectories the policy could produce.

> 👉 Reading the structure: we run the policy, collect trajectories, sum up the reward in each, and average. A larger $J(\theta)$ means the policy collects more reward on average. Our entire goal reduces to one thing: adjust θ to make $J(\theta)$ as large as possible.

The intuition is direct. If we can compute the gradient of J with respect to θ, we can climb it.

In summary, policy gradient methods learn a parameterized policy by maximizing expected return through gradient ascent. The challenge, which the next section solves, is computing that gradient at all.

The log-derivative trick and the policy gradient theorem

We want the gradient of $J(\theta)$, but there is an immediate obstacle. The objective is an expectation over trajectories, and the distribution of those trajectories itself depends on θ.

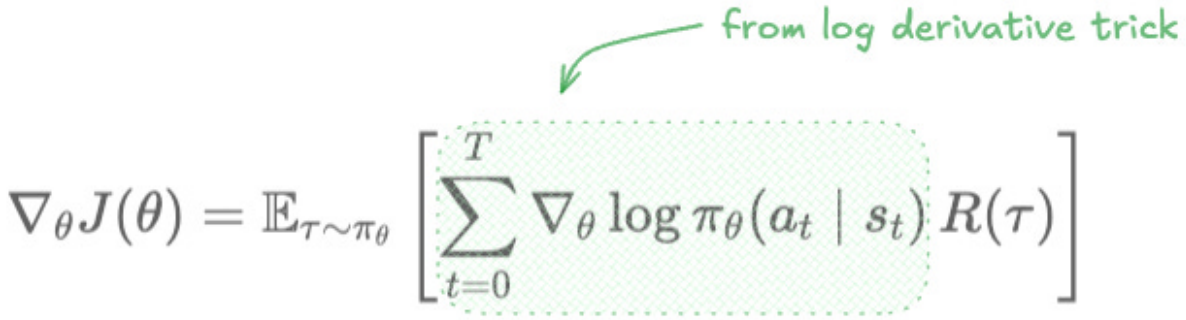
Changing the parameters changes which trajectories we see. We cannot simply differentiate inside the average, because the thing we are averaging over is moving as we differentiate.

This is where one elegant identity rescues us. It is called the log-derivative trick, and it rests on a basic fact from calculus. The derivative of the natural logarithm of a function is the derivative of the function divided by the function itself:

Rearranging it gives the form we actually use: $\nabla_\theta p(x) = p(x)\nabla_\theta \log p(x)$.

Why does this matter? It converts a gradient of a probability into the probability times a gradient of a log-probability. The first form is hard to average over by sampling. The second form is exactly an expectation, which we can estimate by sampling. That single conversion is what makes the whole method work.

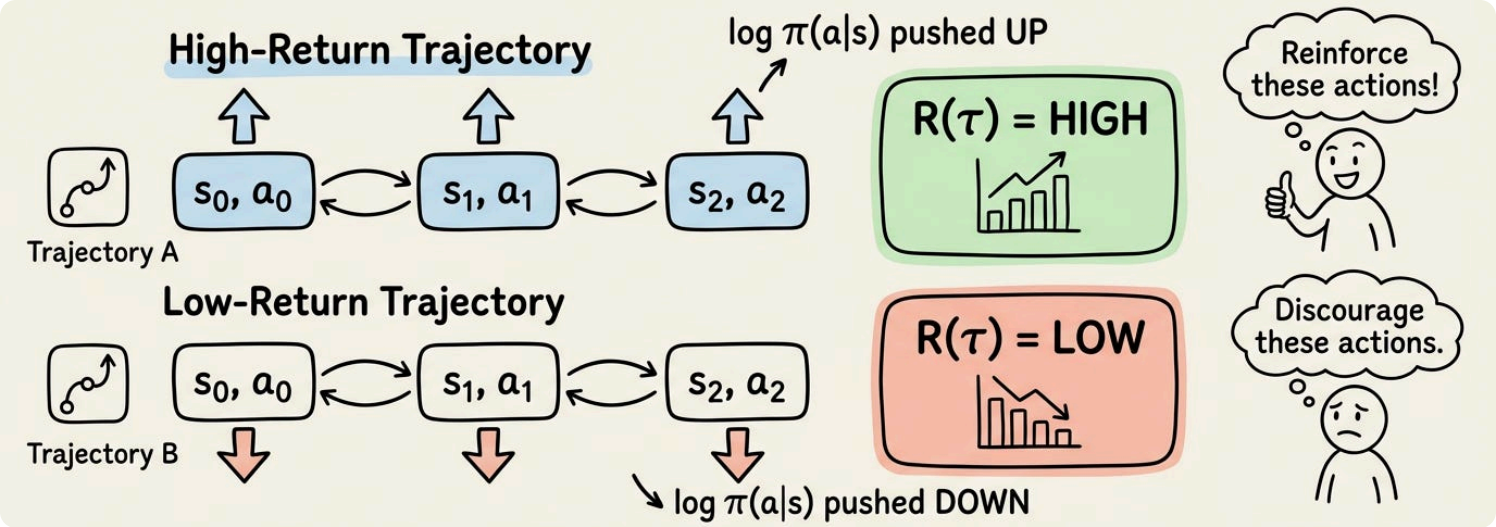
Applying this trick to our objective and working through the algebra of a trajectory's probability gives the central result of the field, the policy gradient theorem:



For each action in the trajectory, we compute how the log-probability of that action would change as we nudge the parameters. We weight that direction by the return of the whole trajectory. Then we sum over the trajectory and average over many trajectories.

The intuition is the heart of the method. The gradient pushes up the probability of actions that led to high return and pushes down the probability of actions that led to low return.

> 👉 If a trajectory earned a large reward, every action in it gets reinforced. If it earned little, every action gets discouraged. Good outcomes make their actions more likely, bad outcomes make their actions less likely.



👉 Important: Policy gradients are model-free.

In summary, the log-derivative trick turns an intractable gradient into a sample-able expectation, and the policy gradient theorem gives us a formula we can estimate from experience alone. The next step is turning this formula into an algorithm.

REINFORCE

The policy gradient theorem tells us what the gradient is. REINFORCE tells us how to estimate it from experience. It was the first policy gradient method.

The technique is direct:

- Run the current policy to collect a full episode.
- For each step, record the action's log-probability and the rewards that followed.
- After the episode ends, compute the return, weight each log-probability by it, and take one gradient ascent step.
- Repeat with a fresh episode.

> 👉 Because REINFORCE waits for a complete episode before computing returns, it is a Monte Carlo method.

In practice, one refinement is almost always applied. An action taken at time t cannot influence rewards that came before it. So instead of weighting each action by the entire trajectory's return, we weight it only by the return collected from that point onward, i.e., G_t.

Thus, we write REINFORCE gradient estimate as the following:

$$\nabla_\theta J(\theta) \approx \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t$$

This is the same shape as the policy gradient theorem, with one change. The trajectory return $R(\tau)$ has been replaced by the return-to-go G_t. Everything else is identical.

> 👉 REINFORCE produces an unbiased estimate of the policy gradient. Unbiased means that if we averaged over infinitely many episodes, the estimate would equal the true gradient exactly. There is no systematic error. This is an important property, and we will contrast it with what comes later.

In summary, REINFORCE is the Monte Carlo policy gradient. It is simple, principled, and unbiased. It is also, as we are about to see, painfully noisy.

The variance problem

REINFORCE is unbiased, which sounds ideal. But unbiased does not mean reliable on any single estimate, it means correct on average. The individual estimates can still scatter wildly around that average, and REINFORCE estimates scatter a great deal.

Consider where the noise comes from. The return G_t is a sum of many rewards across many steps, and each of those rewards depends on a chain of random action choices and random environment transitions. A single lucky or unlucky episode can produce a return far from the typical value.

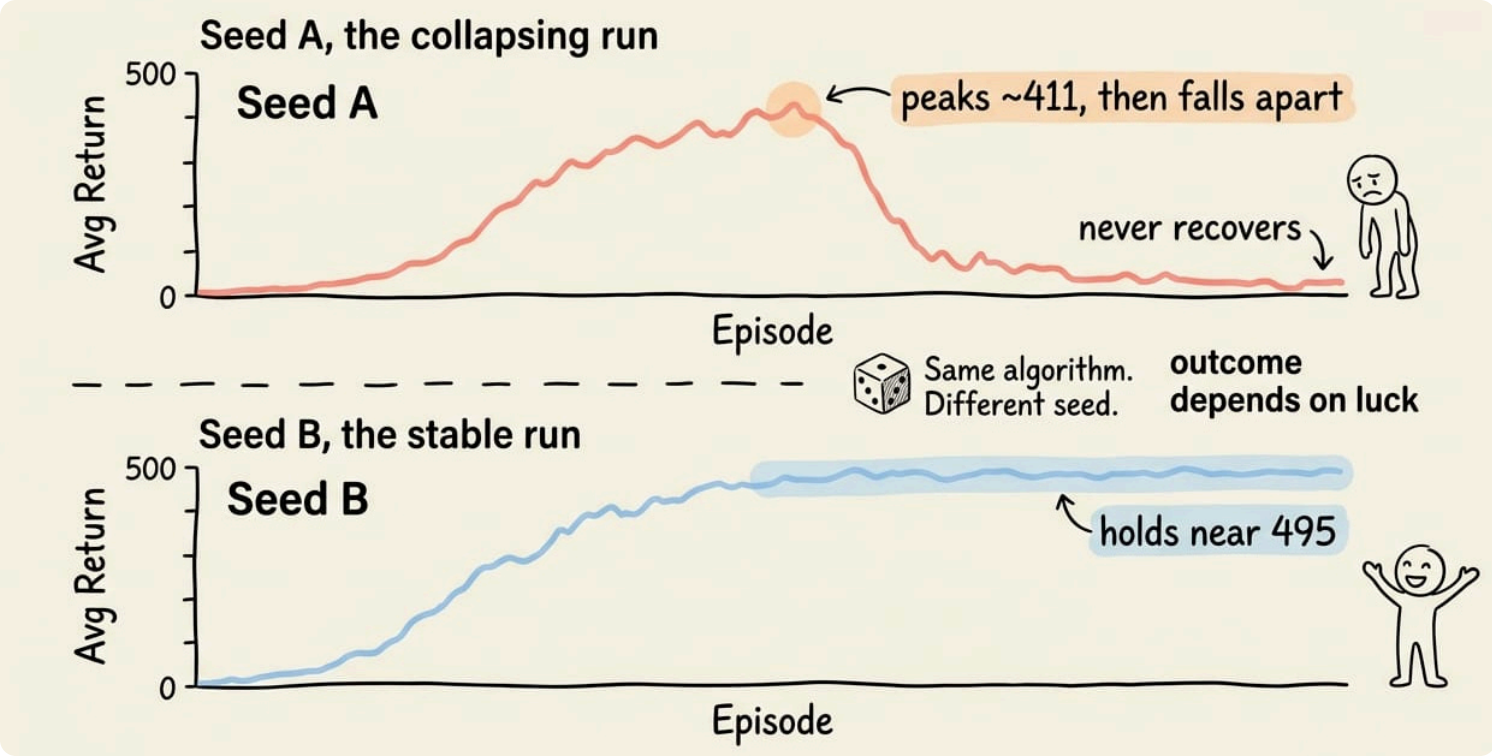
> 👉 Since we weight the gradient by this return, the gradient inherits all of that variability, leading to huge instability from one training run to another.

This is the variance problem. High variance means our gradient estimate jumps around from episode to episode, even when the true gradient is steady. We end

up taking steps in noisy, inconsistent directions. Learning, thus, becomes unstable.

The two most common observations are:

- Because REINFORCE relies entirely on complete experience trajectories, performance becomes highly sensitive to the initial weight initialization and the sequence of randomly sampled episodes. Two identical models running simultaneously with different random seeds can have entirely different trajectories; one might master the task while the other fails to learn at all.



- Training returns climb for hundreds of episodes, then suddenly plummet. High variance is a major contributor: a few noisy gradient steps in the wrong direction undo hard-won progress, and the policy struggles to recover.

👉 A high-variance estimator is not wrong, it is just unreliable per sample. To get a trustworthy gradient from a noisy estimator, you would need to average over many episodes per update, which is exactly why basic REINFORCE is sample-inefficient.

This sets up the central tension of the chapter. We have an unbiased gradient that is too noisy to use well. The rest of the chapter is about reducing that noise, ideally without giving up too much of the unbiasedness. This is a bias-variance trade-off, and we will navigate it step-by-step.

Baselines

Here is the key insight:

We are weighting each action by its return. But a return of 400 does not, on its own, tell us whether 400 was good. If every action from this state tends to yield around 400, then 400 is merely average, and we should not strongly reinforce it. What matters is whether an action did better or worse than expected.

So we subtract a reference value, called a baseline, from the return before weighting. The baseline represents roughly what we expected to get. The modified gradient estimate looks like this:

$$\nabla_\theta J(\theta) \approx \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \, (G_t - b(s_t))$$

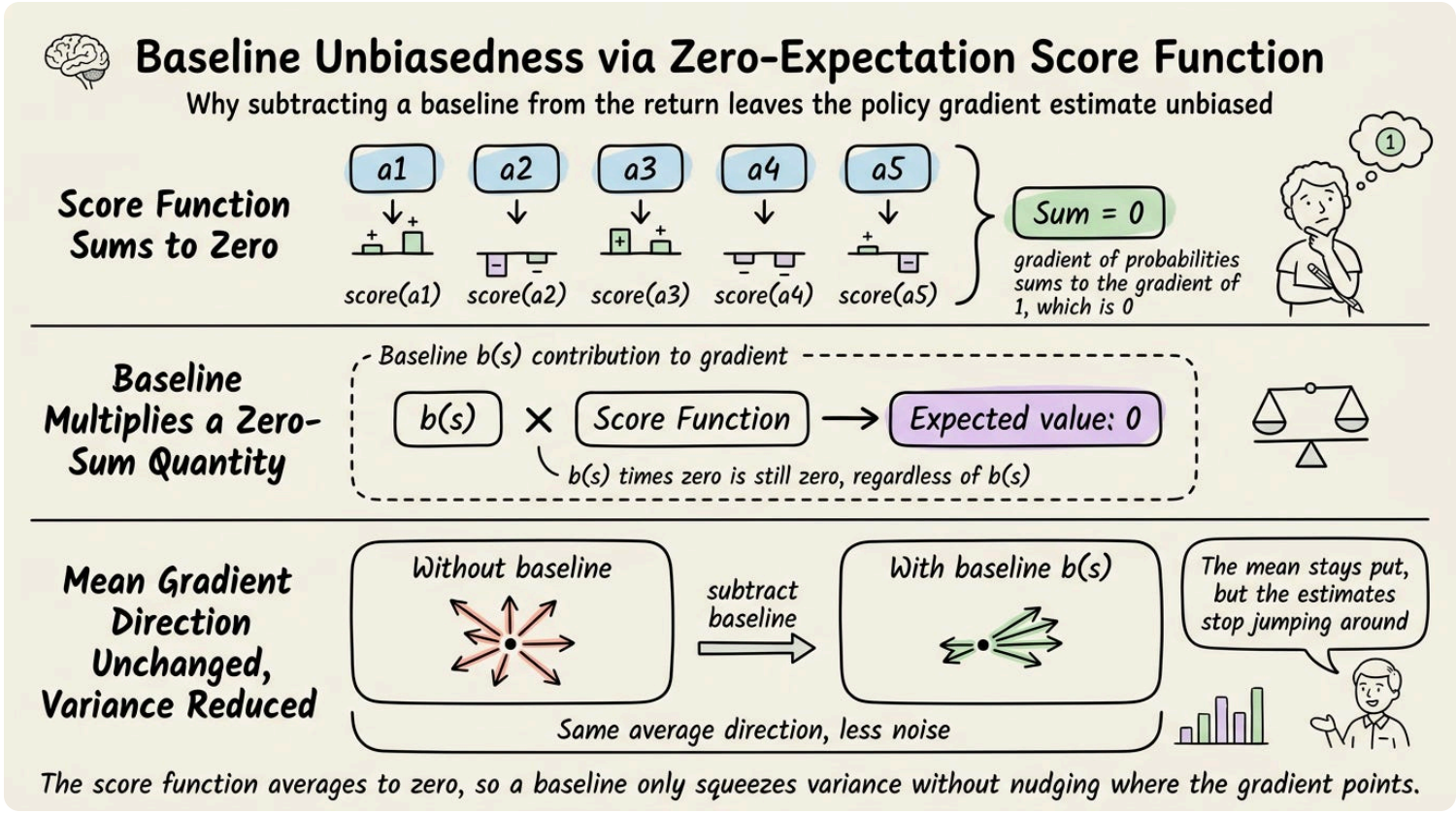
The new term is $b(s_t)$, the baseline evaluated at state s_t. Everything else is the REINFORCE estimate from before. We have simply replaced the raw return G_t with the difference $G_t - b(s_t)$, how much the return exceeded our reference.

The remarkable fact is that subtracting a baseline does not bias the gradient at all, as long as the baseline does not depend on the action. The estimate remains correct on average. This follows from a property of the score function, which states that its expected value is zero:

$$\mathbb{E}_{a \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a \mid s) \right] = 0$$

Let us unpack why this is zero. The expectation is over actions drawn from the policy. Since the score function times the probability is just the gradient of the probability, summing the gradient of the probability over all actions is the gradient of the sum of all probabilities. But probabilities always sum to one, and the gradient of the constant one is zero.

👉 The intuition: since the baseline multiplies the score function, and the score function averages to zero, the baseline's contribution to the gradient averages to zero as well.



This reduces the scatter. By re-centering returns around a typical value, the weights become smaller in magnitude and more balanced between positive and negative. Better than expected actions get a positive push, worse than expected ones get a negative push, and the noisy common scale is removed.

👉 The baseline must not depend on the action taken. A baseline that depends on the state is fine and is what we will use. A baseline that depends on the

action would break the zero-expectation property and bias the gradient.

Thus, subtracting a state-dependent baseline reduces variance without introducing bias. The next natural question is what the best state-dependent baseline is. The answer connects policy gradients back to value functions.

The advantage function

What is the best reference value to subtract at a given state? Intuitively, it should be how much return we expect to collect from that state under our current policy.

That quantity has a name we already know from earlier chapters: the state-value function $V(s)$.

If we use $V(s)$ as our baseline, the term we weight each action by becomes the return minus the value of the state. With the return being single-sample estimate of the true action-value $Q(s, a)$, this difference becomes:

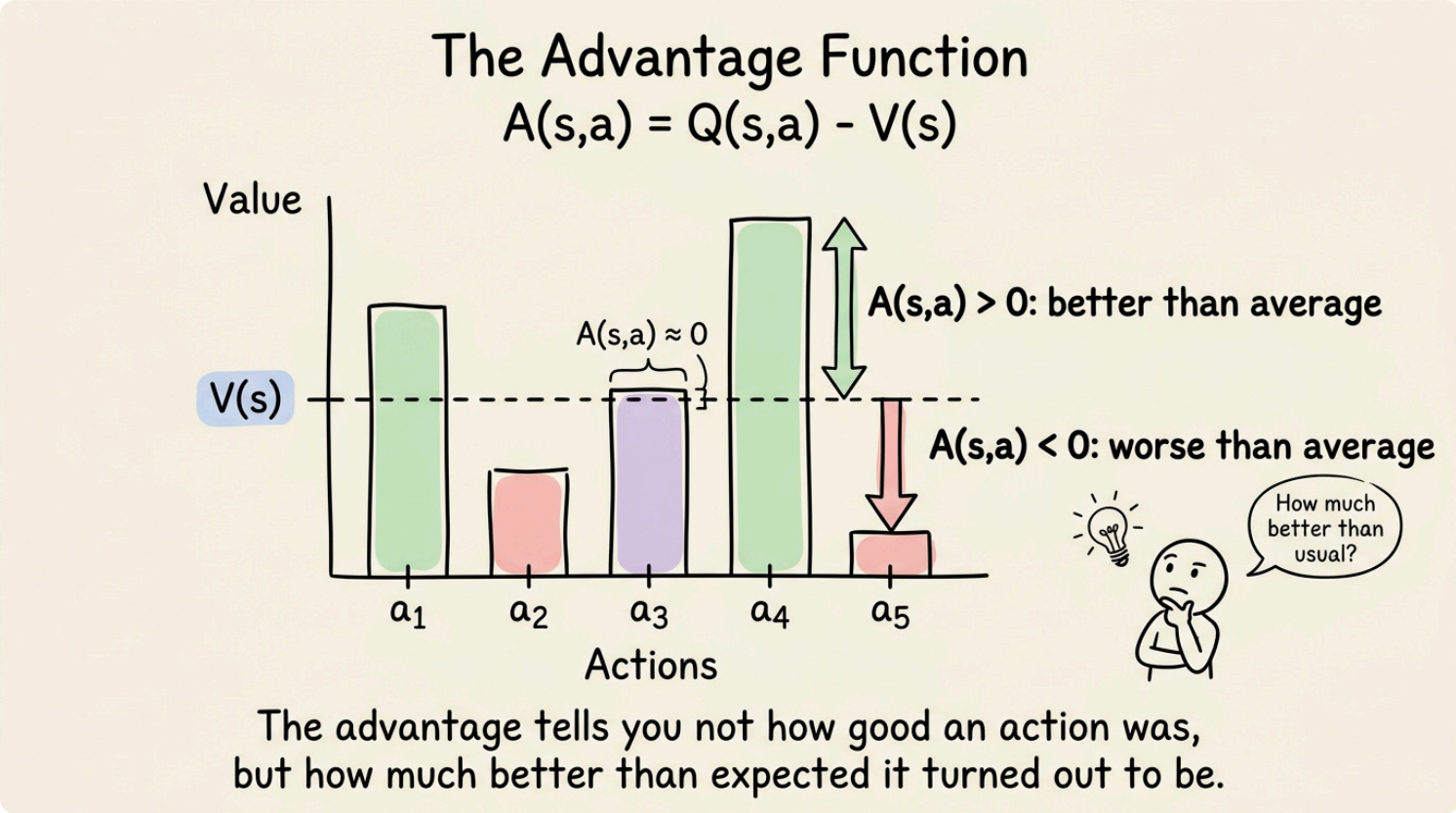
$$A(s, a) = Q(s, a) - V(s)$$

$A(s, a)$ is known as the advantage function.

> 👉 The structure is a comparison: Subtraction gives how much better or worse this particular action (a) is than the policy's typical behavior in that state.

The intuition is clean and worth holding onto. A positive advantage means the action was better than average from this state, so we should make it more likely. A negative advantage means it was worse than average, so we should make it less likely.

The advantage is the cleanest possible learning signal: it measures not how good the outcome was, but how much better than expected.



Substituting the advantage into the policy gradient gives the form used throughout modern reinforcement learning:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \, A(s_t, a_t) \right]$$

This is identical in shape to every policy gradient we have written. We push up actions with positive advantage, push down actions with negative advantage, and the magnitude of the push scales with how far from average the action was.

So overall, the best state-dependent baseline is the value function, and using it produces the advantage function, the signal of how much better than average an action was. Learning that value function is what we turn to now.

Actor-critic architecture

We have now arrived at the actor-critic architecture by following one thread: the best baseline is the value function, but the value function must be learned. So we learn two things at once:

- The first is the actor: the policy $\pi_\theta(a \mid s)$, which chooses actions.
- The second is the critic: a value function $V_\phi(s)$ with its own parameters ϕ, which estimates how good states are.

The actor acts and the critic judges. The actor updates using the critic's judgment as its baseline.

This is why the method is called actor-critic. The critic does not choose actions; it only evaluates them, supplying the advantage estimate that the actor uses to improve.

The two components are trained together but with different goals:

- The actor is trained to maximize expected return, using the advantage-weighted policy gradient we just derived.
- The critic is trained to predict returns accurately, by minimizing the squared difference between its prediction and the observed return.

This critic loss is defined as:

$$\mathcal{L}_{\text{critic}}(\phi) = (V_\phi(s_t) - G_t)^2$$

In this loss, $V_\phi(s_t)$ is the critic's predicted value of the state, G_t is the actual return observed from that state, and ϕ are the critic's parameters.

The squared difference penalizes the critic for predictions that miss the realized return. The structure is ordinary supervised regression: predict a number, compare to the truth, minimize the squared error.

> 👉 The intuition for the whole system is that the critic learns to predict what return to expect from each state, and the actor uses any deviation from that prediction as its signal. When the actor stumbles into a better-than-predicted outcome, the advantage is positive and that behavior is reinforced. The critic, meanwhile, updates its prediction so that next time the bar is set correctly.

Now the most important idea in this section.

The critic is an approximation. It is not the true value function, especially early in training when it is barely trained. So the advantage estimate it produces can be noisy or inaccurate:

$$\hat{A}_t = G_t - V_\phi(s_t)$$

But this does not mean the policy gradient is biased simply because the baseline is approximate. As long as the baseline depends only on the state and not on the action, subtracting it does not change the expected policy-gradient direction. It only changes the variance.

> 👉 Recall the true advantage is defined as $A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$. We usually cannot compute Q directly, but the observed return-to-go G_t is a single-sample estimate of it. Substituting gives the estimator in the above form.

So in this Monte Carlo version, the critic mainly acts as a learned variance-reduction device. A learned $V(s)$ is usually a far better baseline than a single

constant, so the advantage estimates are much tighter, and it achieves this without changing the expected direction of the update. The gradient remains unbiased.

The bias-variance trade-off becomes clearer when the critic is trained with bootstrapping. Instead of regressing toward the full Monte Carlo return G_t, practical actor-critic methods often train the critic against a one-step TD target:

$$r_{t+1} + \gamma V_\phi(s_{t+1})$$

This replaces the full future return with one real reward plus the critic's own estimate of what comes next. That is bootstrapping, and this is where bias is genuinely introduced: the target now depends on the critic's own imperfect prediction, not on actual sampled return. This can be used directly as a low-variance estimate of the advantage.

The same one-step quantity also gives the TD error:

$$\delta_t = r_{t+1} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

This lets the actor update after a single step instead of waiting for the whole episode to finish. The cost is that the learning signal is now partly based on the critic's current estimate of the future.

This is the bias-variance trade-off in its clearest form. Full Monte Carlo returns use only real sampled rewards, so they carry no bias but high variance. One-step TD targets use bootstrapping, so they usually have lower variance but introduce bias. Actor-critic methods live on this spectrum.

> 👉 In code, the actor and critic often share the lower layers of a network and split into two heads near the output. Sharing features can speed learning, but it also means the two losses interact, which is why their relative weighting becomes a tuning knob.

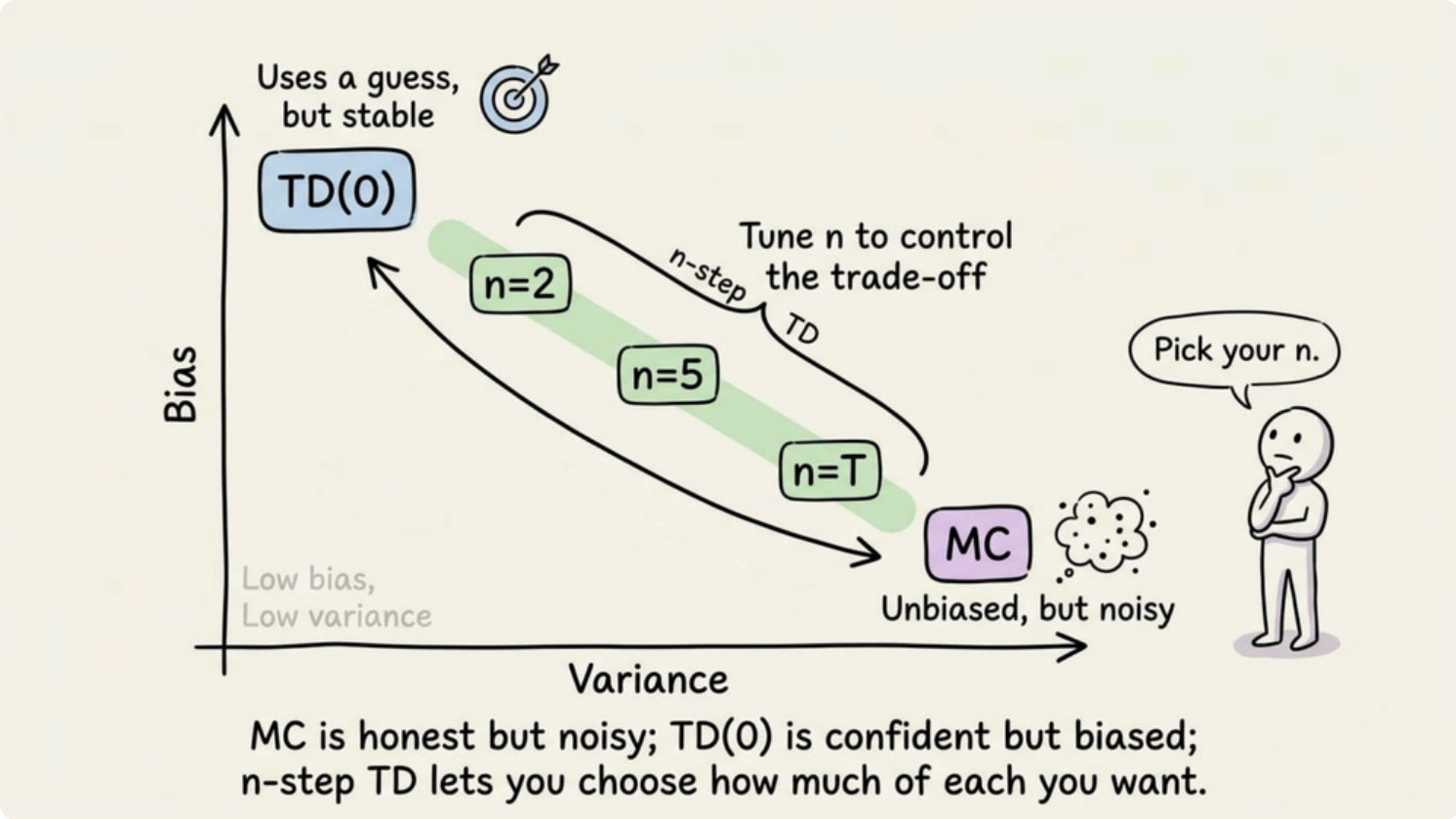
Using more real reward before bootstrapping reduces bias at the cost of variance; using less real reward and bootstrapping earlier reduces variance at the cost of bias. The full Monte Carlo return sits at one end of this spectrum, and

one-step TD sits at the other. This spectrum is exactly what the next section generalizes.

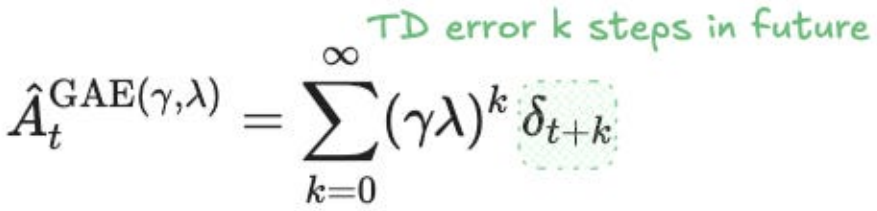
Generalized advantage estimation

We are left with an awkward choice. The one-step TD advantage has low variance but leans heavily on the imperfect critic, giving it bias. The full Monte Carlo advantage has no such bias but high variance. In between sit the n-step estimates, which use n real rewards before bootstrapping from the critic.

Each point on this spectrum has its own bias-variance balance, and picking one means committing to a single compromise.



Generalized advantage estimation (GAE), removes the need to choose. Instead of selecting one estimate, it blends all of them together with exponentially decaying weights:



The symbols:

- $\hat{A}_t^{\text{GAE}}$ is the GAE advantage estimate at time t.
- The term δ_{t+k} is the TD error k steps into the future, which, written with timestep indices, is $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$.
- The discount γ.
- The new parameter λ is a number between zero and one that controls the decay.
- The sum runs over all future steps, with each TD error weighted by $(\gamma\lambda)^k$, so errors further ahead count for geometrically less.

The structure is an exponentially weighted average of TD errors. Near-term errors dominate; distant ones fade. The single knob λ sets how fast they fade, and that knob is the bias-variance dial made continuous.

The intuition lives at the extremes:

- When $\lambda = 0$, only the first TD error survives, and GAE collapses to the one-step estimate: low variance, higher bias.
- When $\lambda = 1$, the weights stop decaying beyond the discount, and GAE becomes the full Monte Carlo advantage: high variance, low bias.
- Any value in between smoothly interpolates. Rather than choosing a discrete n, we turn a continuous dial.

> 👉 GAE is a core ingredient of Proximal Policy Optimization (PPO), the algorithm most associated with tuning language models. When we reach PPO in a later chapter, GAE will already be familiar.

In summary, GAE generalizes the n-step advantage into a single exponentially-weighted estimator, with λ tuning the bias-variance balance continuously. With this, we have a full toolkit for low-variance policy gradients. It is time to see where all of this leads.

The connection to language models

Everything in this chapter underlies how large language models are tuned with reinforcement learning, and the mapping is exact.

In the RL setting:

A language model is a policy. Given the text so far, it outputs a probability distribution over the next token and samples from it. The state s is the prompt plus the tokens generated so far, the action a is the next token and $\pi_\theta(a \mid s)$ is the model itself.

> 👉 Generating a response is rolling out a trajectory, sampling token after token until the sequence ends.

This also answers a question the chapter opened with: why learn a policy directly at all, instead of values? Here the answer is unavoidable. The action space is the entire vocabulary, tens of thousands of tokens. The value-based method, estimates a value for every action and takes the $\arg\max$, which is ugly at the scale of LMs.

A model that directly emits token probabilities sidesteps the problem entirely, because it is a policy by construction. Policy gradients are not one option among several for language models. They are essentially the exact option, which is why this chapter, is also the foundation for modern LLM tuning.

The variance problem we spent this chapter fighting is acute in this setting. Trajectories are long, often hundreds of tokens, and the action space is enormous. A naive REINFORCE-style estimate would be hopelessly noisy. The baseline, the advantage function, and GAE are not optional niceties here; they are what make the training stable enough to work at all.

We will not go in depth or build the full pipeline here. The greater and deeper details are for later chapters. But the engine underneath all of it is already in your hands. It is the policy gradient we studied here.

We have now understood all the key concepts for this chapter. The pieces are in place. So let's go ahead and dive into hands-on section for some practical experimentation.

Hands-on: REINFORCE on CartPole

We will now implement REINFORCE and watch the variance issue play out in real training.

The code and project setup are attached below as a zip file. You can extract it and run `uv sync` to get going.

```
.python-version
pyproject.toml
reinforce_seeds.png
reinforce.py
uv.lock
```

Download the zip file below:

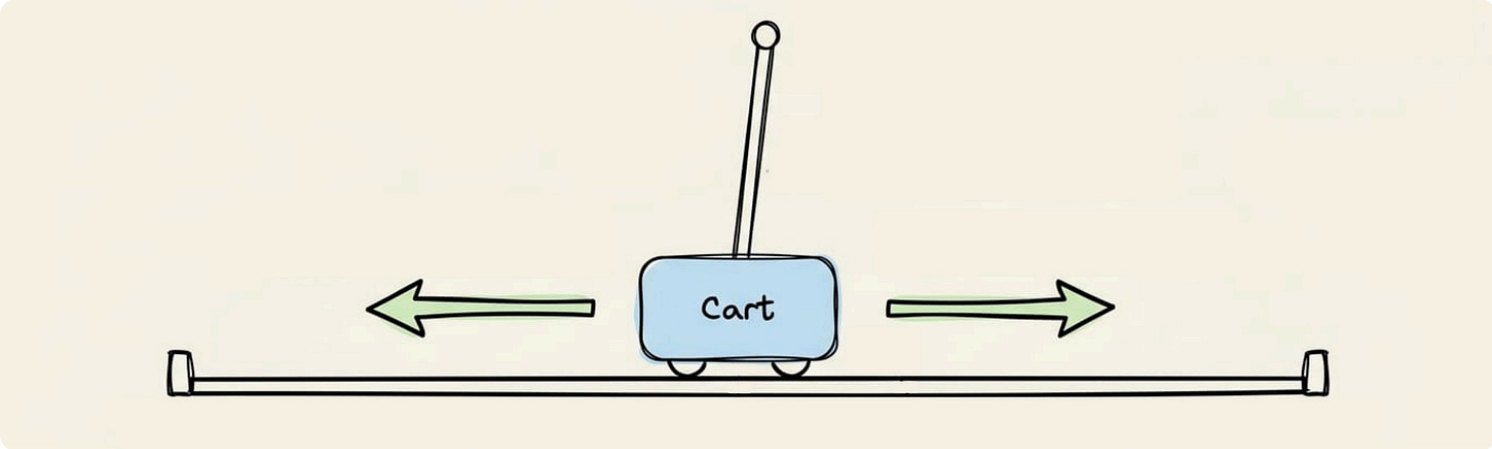
reinforce

reinforce.zip • 203 KB

For details about versions and dependencies, check the `.python-version` and `pyproject.toml` files.

The environment

We use CartPole-v1, the same environment as Chapter 6, so the comparison is grounded in something familiar. An episode return of 500 is the maximum.



REINFORCE

Firstly let's import required libraries and set our variables and hyperparameters:

Now let's define a small policy network, collects one episode at a time, computes the return-to-go for each step, and takes a single gradient ascent step per episode:

This is REINFORCE in its purest form: no baseline, no normalization, just the Monte Carlo policy gradient exactly as the theorem states it.

Walking through what the code does:

- It defines `PolicyNet`, a two-layer network that maps a state to action logits. The logits are left raw because the categorical distribution applies the softmax internally when we sample.
- `discounted_returns` computes the return-to-go for every step, accumulating rewards backward through time with the discount factor so each value is found in a single pass.
- The `run` function holds the training loop. Inside its `for ep in range(EPISODES)` loop, each iteration handles one episode: it resets the environment, then runs a full rollout, at each step sampling an action from the policy's categorical distribution, storing that action's log-probability (the score function), stepping the environment, and recording the reward, until the episode ends.

> 👉 Collecting the whole episode before updating is what makes this a Monte Carlo method.

- After the episode, the loss is the negative sum over timesteps of each log-probability weighted by its return-to-go. The sum is the theorem's within-

trajectory term; the single episode is an unbiased one-sample estimate of the expectation over trajectories.

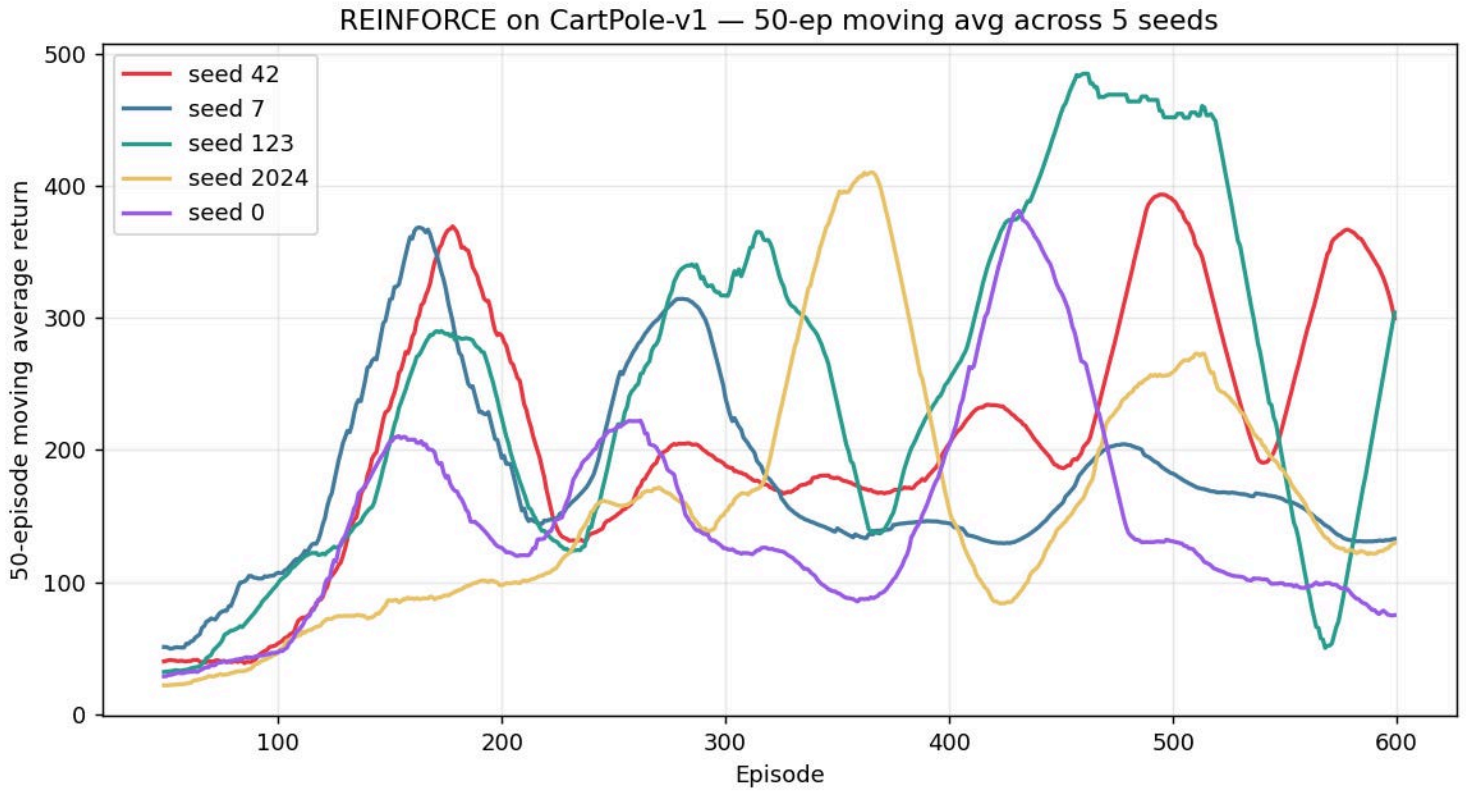
> 👉 The negative is only because optimizers minimize while we want to ascend.

- Then `run` does one backward pass and one optimizer step to update the policy, logs the episode's total reward to `history`, and moves to the next episode. No baseline, no normalization, this is raw REINFORCE, which is why the variance problem is so visible when we plot the results.
- `moving_avg` is a plotting helper that smooths the noisy per-episode returns with a sliding window so the learning trend is readable.

Finally, let's take a look at the driver code:

We run it across five random seeds to see how the algorithm behaves run-to-run and generate the plot.

Here is the plot we got on running the code:



👉 The exact curves can vary across machines, but the behavior would mostly be consistent.

The plot is the variance problem made visible. Every seed climbs somewhere or the other, but not able to holds its gains. Each curve oscillates violently, peaking, crashing back toward 100, then climbing again, throughout all 600 episodes.

This is not a bug. It is exactly what the theory predicts. Vanilla REINFORCE weights every action by the noisy Monte Carlo return G_t, and a single unlucky moment can produce a gradient that pushes the policy off a hard-won peak.

The five seeds also disagree wildly with one another, which is the same variance viewed across runs rather than across time.

Now, as a self-learning exercise, readers are encouraged to extend this in a few directions:

- Add a simple baseline.
- Then add a learned value baseline to form an actor-critic, using the TD error as the advantage.
- Vary the learning rate and observe the different plots.

With that, the hands-on section concludes, and so does this chapter. In the upcoming chapters, we will continue to build on the core ideas of RL and explore concepts and their implementations wherever applicable.

Conclusion

In this chapter, we explored policy gradient methods, the family that learns behavior directly.

We began by contrasting value-based and policy-based control, and parameterized the policy directly to maximize expected return.

We then built the policy gradient from first principles: the log-derivative trick turned an intractable gradient into a sample-able expectation, and the policy gradient theorem gave us a formula estimable from experience, with the environment dynamics dropping out entirely.

REINFORCE turned that formula into the first policy gradient algorithm, simple and unbiased.

We then confronted its central weakness, high variance. The fix unfolded in stages:

- A baseline reduced variance without introducing bias.
- Then the best state-dependent baseline turned out to be the value function, which gave us the advantage function, the signal of how much better than

average an action was.

- Learning that value function alongside the policy produced the actor-critic architecture, reducing variance through a learned baseline and with bootstrapping, it trades some bias for further variance reduction.

We then connected the whole chapter to language models, where the model is a policy over tokens and human-feedback (RLHF) tuning is advantage-based policy gradients at scale. We will be diving deeper into these topics in the later chapters.

Finally, the hands-on section implemented the REINFORCE algorithm and trained on CartPole.

In the next chapter, we will build on the foundations and learn about proximal policy optimization.

The aim, as ever, is to develop a solid system-level perspective and to equip you with an adaptable engineering framework for building AI systems that are robust and maintainable.

As always, thanks for reading!

Any questions?

Feel free to post them in the comments.

Or

if you wish to connect privately, feel free to initiate a chat here:

Previous

Introduction to Deep RL and DQN